

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 3.1
по дисциплине: Дискретная математика
по теме: Отношения

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель

Отношения и их свойства

Цель работы: изучить способы задания отношений, операции над отношениями и свойства отношений, научиться программно реализовывать операции и определять свойства отношений.

Задания

Часть 1. Операции над отношениями

- 1.1. Представить отношения (см. "Варианты заданий", п.а) графиком, графом и матрицей.
- 1.2. Вычислить значение выражения (см. "Варианты заданий", п.б) при заданных отношениях (см. "Варианты заданий", п.а).
- 1.3. Написать программы, формирующие матрицы заданных отношений (см. "Варианты заданий", п.а).
- 1.4. Программно реализовать операции над отношениями.
- 1.5. Написать программу, вычисляющую значение выражения (см. "Варианты заданий", п.б) и вычислить его при заданных отношениях (см. "Варианты заданий", п.а).

Задание 1.1) В тетради

Задание 1.2) В тетради

Задание № 1.3. Написать программы, формирующие матрицы заданных отношений (см. "Варианты заданий", п.а).

Матрица для отношения A :

$A = \{(x, y) | x \in \mathbb{N}, y \in \mathbb{N}, x < 11, y < 11, x + y \text{ кратно трём}\}$

Код программы :

```
#include <iostream>
#include <set>
#include <vector>

using namespace std;

typedef pair<int, int> Pair;

int main() {
    set<Pair> A;

    for (int x = 1; x < 11; ++x) {
        for (int y = 1; y < 11; ++y) {
            if ((x + y) % 3 == 0) {
                A.insert(make_pair(x, y));
            }
        }
    }

    int maxX = 0, maxY = 0;
    for (const auto &pair: A) {
        maxX = max(maxX, pair.first);
        maxY = max(maxY, pair.second);
    }

    vector<vector<int>> matrix(maxX, vector<int>(maxY, 0));

    for (const auto &pair: A) {
        int x = pair.first;
        int y = pair.second;
        matrix[x - 1][y - 1] = 1;
    }

    cout << "Relationship Matrix A:" << endl;

    cout << " ";
    cout << " ";

    for (int j = 1; j <= maxY; ++j) {
        cout << j << " " << "| ";
    }
    cout << endl;

    for (int i = 0; i < maxX; ++i) {
        cout << i + 1 ;
        if (i + 1 < 10) {
            cout << " " << "|";
        } else {
            cout << " |";
        }

        for (int j = 0; j < maxY; ++j) {
            cout << matrix[i][j] << " | ";
        }

        cout << endl;
    }

    return 0;
}
```

Вывод программы :

Relationship Matrix A:											
	1	2	3	4	5	6	7	8	9	10	
1	0	1	0	0	1	0	0	1	0	0	
2	1	0	0	1	0	0	1	0	0	1	
3	0	0	1	0	0	1	0	0	1	0	
4	0	1	0	0	1	0	0	1	0	0	
5	1	0	0	1	0	0	1	0	0	1	
6	0	0	1	0	0	1	0	0	1	0	
7	0	1	0	0	1	0	0	1	0	0	
8	1	0	0	1	0	0	1	0	0	1	
9	0	0	1	0	0	1	0	0	1	0	
10	0	1	0	0	1	0	0	1	0	0	

Матрица для отношения B :

$B = \{(x, y) | x \in \mathbb{N}, y \in \mathbb{N}, x < 11, y < 11, (2 < x < 8) \text{ или } (2 < y < 8)\}$

Код программы :

```
#include <iostream>
#include <set>
#include <vector>

using namespace std;

typedef pair<int, int> Pair;

int main() {
    set<Pair> B;

    for (int x = 3; x < 8; ++x) {
        for (int y = 3; y < 8; ++y) {
            B.insert(make_pair(x, y));
        }
    }

    int maxX = 0, maxY = 0;
    for (const auto &pair: B) {
        maxX = max(maxX, pair.first);
        maxY = max(maxY, pair.second);
    }

    vector<vector<int>> matrix(maxX, vector<int>(maxY, 0));

    for (const auto &pair: B) {
        int x = pair.first;
        int y = pair.second;
        matrix[x - 1][y - 1] = 1;
    }

    cout << "Relationship Matrix B:" << endl;

    cout << " ";
    cout << " ";

    for (int j = 1; j <= maxY; ++j) {
        cout << j << " " << "| ";
    }
    cout << endl;

    for (int i = 1; i <= maxX; ++i) {
        cout << i << " " << "| ";

        for (int j = 0; j <= maxY; ++j) {
            cout << matrix[i][j] << " | ";
        }

        cout << endl;
    }

    return 0;
}
```

Вывод программы :

Relationship Matrix B:								
	1	2	3	4	5	6	7	
1	0	0	0	0	0	0	0	
2	0	0	1	1	1	1	1	
3	0	0	1	1	1	1	1	
4	0	0	1	1	1	1	1	
5	0	0	1	1	1	1	1	
6	0	0	1	1	1	1	1	

Матрица для отношения C :

$C = \{(x, y) | x \in \mathbb{N}, y \in \mathbb{N}, x < 11, y < 11, x^2 + y^2 < 100\}$

Код программы :

```
#include <iostream>
#include <set>
#include <vector>

using namespace std;

typedef pair<int, int> Pair;

int main() {
    set<Pair> C;

    for (int x = 1; x < 11; ++x) {
        for (int y = 1; y < 11; ++y) {
            if ((x * x + y * y) < 100) {
                C.insert(make_pair(x, y));
            }
        }
    }

    int maxX = 0, maxY = 0;
    for (const auto &pair: C) {
        maxX = max(maxX, pair.first);
        maxY = max(maxY, pair.second);
    }

    vector<vector<int>> matrix(maxX, vector<int>(maxY, 0));

    for (const auto &pair: C) {
        int x = pair.first;
        int y = pair.second;
        matrix[x - 1][y - 1] = 1;
    }

    cout << "Relationship Matrix C:" << endl;

    cout << " ";
    cout << " ";

    for (int j = 1; j <= maxY; ++j) {
        cout << j << " " << "| ";
    }
    cout << endl;

    for (int i = 0; i < maxX; ++i) {
        cout << i + 1 ;
        if (i + 1 < 10) {
            cout << " " << "|";
        } else {
            cout << " |";
        }

        for (int j = 0; j < maxY; ++j) {
            cout << matrix[i][j] << " | ";
        }

        cout << endl;
    }

    return 0;
}
```

Вывод программы :

Relationship Matrix C:										
	1	2	3	4	5	6	7	8	9	
1	1	1	1	1	1	1	1	1	1	1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	1	1	1	1	1	1	1
4	1	1	1	1	1	1	1	1	1	1
5	1	1	1	1	1	1	1	1	0	0
6	1	1	1	1	1	1	1	0	0	0
7	1	1	1	1	1	1	1	0	0	0
8	1	1	1	1	1	0	0	0	0	0
9	1	1	1	1	0	0	0	0	0	0

Задание №1.4 Программно реализовать операции над отношениями.

```
#include <iostream>
#include <set>
#include <vector>

using namespace std;

typedef set<pair<int, int>> Attitude;

void printRelation(const Attitude& rel) {
    for (const auto& pair : rel) {
        cout << "(" << pair.first << "," << pair.second << ") ";
    }
    cout << endl;
}

bool isSubset(const Attitude& A, const Attitude& B) {
    for (const auto& pair : A) {
        if (B.find(pair) == B.end()) {
            return false;
        }
    }
    return true;
}

bool isEqual(const Attitude& A, const Attitude& B) {
    return isSubset(A, B) && isSubset(B, A);
}

Attitude unionRelations(const Attitude& A, const Attitude& B) {
    Attitude result = A;
    for (const auto& pair : B) {
        result.insert(pair);
    }
    return result;
}

Attitude intersectRelations(const Attitude& A, const Attitude& B) {
    Attitude result;
    for (const auto& pair : A) {
        if (B.find(pair) != B.end()) {
            result.insert(pair);
        }
    }
    return result;
}

Attitude subtractRelations(const Attitude& A, const Attitude& B) {
    Attitude result = A;
    for (const auto& pair : B) {
        result.erase(pair);
    }
    return result;
}

Attitude symmetricDifference(const Attitude& A, const Attitude& B) {
    Attitude result = unionRelations(subtractRelations(A, B),
    subtractRelations(B, A));
    return result;
}
```

```

Attitude composeRelations(const Attitude& A, const Attitude& B) {
    Attitude result;
    for (const auto& pairA : A) {
        int xA = pairA.first;
        int yA = pairA.second;
        for (const auto& pairB : B) {
            int xB = pairB.first;
            int yB = pairB.second;
            if (yA == xB) {
                result.insert({xA, yB});
            }
        }
    }
    return result;
}

Attitude reverseRelation(const Attitude& A) {
    Attitude result;
    for (const auto& pair : A) {
        result.insert({pair.second, pair.first});
    }
    return result;
}

Attitude complementRelation(const Attitude& A, int universeSize) {
    Attitude result;
    for (int i = 1; i <= universeSize; ++i) {
        for (int j = 1; j <= universeSize; ++j) {
            if (A.find({i, j}) == A.end()) {
                result.insert({i, j});
            }
        }
    }
    return result;
}

int main() {
    Attitude A = {{1, 2}, {2, 3}, {3, 4}};
    Attitude B = {{2, 3}, {4, 5}, {5, 6}};

    cout << "Attitude A: ";
    printRelation(A);

    cout << '\n';

    cout << "Attitude B: ";
    printRelation(B);

    cout << "-----\n";

    if (isSubset(A, B)) {
        cout << "A included in B." << endl;
    } else {
        cout << "A not included in B." << endl;
    }

    cout << "-----\n";

    if (isEqual(A, B)) {
        cout << "A and B equal." << endl;
    } else {
        cout << "A and B not equal." << endl;
    }
}

```

```

        cout << "-----\n";

        Attitude unionResult = unionRelations(A, B);
        cout << "An association A and B: ";
        printRelation(unionResult);

        cout << "-----\n";

        Attitude intersectionResult = intersectRelations(A, B);
        cout << "Intersection A and B: ";
        printRelation(intersectionResult);

        cout << "-----\n";

        Attitude subtractionResult = subtractRelations(A, B);
        cout << "Difference A and B: ";
        printRelation(subtractionResult);

        cout << "-----\n";

        Attitude symmetricDiffResult = symmetricDifference(A, B);
        cout << "Symmetric difference A and B: ";
        printRelation(symmetricDiffResult);
        Attitude compositionResult = composeRelations(A, B);

        cout << "-----\n";

        cout << "Composition A and B: ";
        printRelation(compositionResult);

        cout << "-----\n";

        Attitude reversedResultA = reverseRelation(A);
        cout << "Converted attitude A: ";
        printRelation(reversedResultA);

        cout << "-----\n";

        int universeSize = 6;
        Attitude complementResultA = complementRelation(A, universeSize);
        cout << "Addition A to a universal relation: ";
        printRelation(complementResultA);

        return 0;
}

```

Результат работа программы

```

Attitude A: (1,2) (2,3) (3,4)

Attitude B: (2,3) (4,5) (5,6)
-----
A not included in B.
-----
A and B not equal.
-----
An association A and B: (1,2) (2,3) (3,4) (4,5) (5,6)
-----
Intersection A and B: (2,3)
-----
Difference A and B: (1,2) (3,4)
-----
Symmetric difference A and B: (1,2) (3,4) (4,5) (5,6)
-----
Composition A and B: (1,3) (3,5)
-----
Converted attitude A: (2,1) (3,2) (4,3)
-----
Addition A to a universal relation: (1,1) (1,3) (1,4) (1,5) (1,6) (2,1) (2,2) (2,4) (2,5) (2,6) (3,1) (3,2) (3,3) (3,5)
(3,6) (4,1) (4,2) (4,3) (4,4) (4,5) (4,6) (5,1) (5,2) (5,3) (5,4) (5,5) (5,6) (6,1) (6,2) (6,3) (6,4) (6,5) (6,6)

```

Задание №1.5 Написать программу, вычисляющую значение выражения (см. “Варианты заданий”, п.б) и вычислить его при заданных отношениях(см.”Варианты заданий”, п.а).

```
#include <iostream>
#include <set>
#include <vector>

using namespace std;

typedef set<pair<int, int>> Attitude;

void printRelation(const Attitude& rel) {
    for (const auto& pair : rel) {
        cout << "(" << pair.first << "," << pair.second << ") ";
    }
    cout << endl;
}

Attitude unionRelations(const Attitude& A, const Attitude& B) {
    Attitude result = A;
    for (const auto& pair : B) {
        result.insert(pair);
    }
    return result;
}

Attitude differenceRelations(const Attitude& A, const Attitude& B) {
    Attitude result = A;
    for (const auto& pair : B) {
        result.erase(pair);
    }

    return result;
}

Attitude composeRelations(const Attitude& A, const Attitude& B) {
    Attitude result;
    for (const auto& pairA : A) {
        int xA = pairA.first;
        int yA = pairA.second;
        for (const auto& pairB : B) {
            int xB = pairB.first;
            int yB = pairB.second;
            if (yA == xB) {
                result.insert({xA, yB});
            }
        }
    }

    return result;
}

Attitude reverseRelation(const Attitude& A) {
    Attitude result;
    for (const auto& pair : A) {
        result.insert({pair.second, pair.first});
    }
    return result;
}
```

```

int main() {
    Attitude A;

    for (int x = 1; x < 11; ++x) {
        for (int y = 1; y < 11; ++y) {
            if ((x + y) % 3 == 0) {
                A.insert(make_pair(x, y));
            }
        }
    }

    Attitude B;

    for (int x = 3; x < 8; ++x) {
        for (int y = 3; y < 8; ++y) {
            B.insert(make_pair(x, y));
        }
    }

    Attitude C;

    for (int x = 1; x < 11; ++x) {
        for (int y = 1; y < 11; ++y) {
            if ((x * x + y * y) < 100) {
                C.insert(make_pair(x, y));
            }
        }
    }

    Attitude A2 = composeRelations(A, A);

    Attitude AInMinusTheFirst = reverseRelation(A);

    Attitude compose = composeRelations(AInMinusTheFirst, C);

    Attitude dif = differenceRelations(A2, B);

    Attitude D = unionRelations(compose, dif);

    printRelation(D);

    return 0;
}

```

Результат работы программы:

```

(1,1) (1,2) (1,3) (1,4) (1,5) (1,6) (1,7) (1,8) (1,9) (1,10) (2,1) (2,2) (2,3) (2,4) (2,5) (2,6) (2,7) (2,8) (2,9) (3,1)
(3,2) (3,3) (3,4) (3,5) (3,6) (3,7) (3,8) (3,9) (4,1) (4,2) (4,3) (4,4) (4,5) (4,6) (4,7) (4,8) (4,9) (4,10) (5,1) (5,2)
(5,3) (5,4) (5,5) (5,6) (5,7) (5,8) (5,9) (6,1) (6,2) (6,3) (6,4) (6,5) (6,6) (6,7) (6,8) (6,9) (7,1) (7,2) (7,3) (7,4)
(7,5) (7,6) (7,7) (7,8) (7,9) (7,10) (8,1) (8,2) (8,3) (8,4) (8,5) (8,6) (8,7) (8,8) (8,9) (9,1) (9,2) (9,3) (9,4) (9,
5) (9,6) (9,7) (9,8) (9,9) (10,1) (10,2) (10,3) (10,4) (10,5) (10,6) (10,7) (10,8) (10,9) (10,10)

```